



Experiment 05

Conditional Aggregation & Multi-Condition Rate Analysis in SQL

Name: Deven

UID: 24BCF10060

Aim

To study and implement advanced SQL techniques such as `LEFT JOIN` based rate calculations, conditional aggregation using `CASE`, and multi-condition filtering with date ranges to solve real-world business analytics problems.

Question 1 – User Confirmation Rate (Medium)

A website maintains records of users who have registered on the platform and the status of confirmation requests sent to them. A confirmation request can either be successfully completed (`confirmed`) or may expire without completion (`timeout`).

For each registered user, calculate the *confirmation rate*, defined as:

$$\text{Confirmation Rate} = \frac{\text{Number of confirmed requests}}{\text{Total number of requests}}$$

If a user has never received any confirmation request, their confirmation rate should be considered 0. Display the user ID along with the confirmation rate rounded to 2 decimal places. The result can be returned in any order.

Example: A user has the following confirmation records: `confirmed`, `confirmed`, `timeout`. Successful Confirmations = 2, Total Requests = 3, so Confirmation Rate = $2/3 = 0.67$.

Given Data

```
1 CREATE TABLE Signups
2 (
3     user_id    INT PRIMARY KEY,
4     time_stamp TIMESTAMP
5 );
6
7 CREATE TABLE Confirmations
8 (
9     user_id    INT,
10    time_stamp  TIMESTAMP,
11    action      VARCHAR(10),
12    FOREIGN KEY(user_id) REFERENCES Signups(user_id)
13 );
14
15 INSERT INTO Signups VALUES
16 (3, '2020-03-21 10:16:13'),
17 (7, '2020-01-04 13:57:59'),
18 (2, '2020-07-29 23:09:44'),
19 (6, '2020-12-09 10:39:37');
20
21 INSERT INTO Confirmations VALUES
22 (3, '2021-01-06 03:30:46', 'timeout'),
```



```
23 (3, '2021-07-14 14:00:00', 'timeout'),
24 (7, '2021-06-12 11:57:29', 'confirmed'),
25 (7, '2021-06-13 12:58:28', 'confirmed'),
26 (7, '2021-06-14 13:59:27', 'confirmed'),
27 (2, '2021-01-22 00:00:00', 'confirmed'),
28 (2, '2021-02-28 23:59:59', 'timeout');
```

Answer

```
1 SELECT
2 s.user_id,
3
4 ROUND(
5     COALESCE(
6         SUM(CASE WHEN c.action = 'confirmed' THEN 1 ELSE 0 END)::
7             NUMERIC
8         / NULLIF(COUNT(c.action), 0),
9     , 2) AS confirmation_rate
10
11 FROM Signups s
12
13 LEFT JOIN Confirmations c
14 ON s.user_id = c.user_id
15
16 GROUP BY s.user_id;
```

Output

user_id	confirmation_rate
2	0.50
3	0.00
6	0.00
7	1.00



Question 2 – Trips and Users Cancellation Rate (Hard)

A ride-booking company maintains information about trip requests and the users involved in those trips. Some users may be restricted (banned) from using the platform, while others are active participants. Each trip can either be completed successfully or cancelled by one of the participants.

For each day between 2013-10-01 and 2013-10-03, determine the proportion of cancelled trips among all valid trips:

$$\text{Cancellation Rate} = \frac{\text{Cancelled Trips}}{\text{Total Valid Trips}}$$

A trip is considered valid only when *both* the client and the driver associated with it are active (not banned) users. The final result should display the date and the calculated cancellation rate rounded to two decimal places.

Given Data

```
1 CREATE TABLE Users
2 (
3     users_id INT PRIMARY KEY,
4     banned   VARCHAR(5),
5     role     VARCHAR(10)
6 );
7
8 CREATE TABLE Trips
9 (
10     id          INT PRIMARY KEY,
11     client_id   INT,
12     driver_id   INT,
13     city_id     INT,
14     status      VARCHAR(25),
15     request_at  DATE,
16     FOREIGN KEY(client_id) REFERENCES Users(users_id),
17     FOREIGN KEY(driver_id) REFERENCES Users(users_id)
18 );
19
20 INSERT INTO Users VALUES
21 (1, 'No', 'client'),
22 (2, 'Yes', 'client'),
23 (3, 'No', 'client'),
24 (4, 'No', 'client'),
25 (10, 'No', 'driver'),
26 (11, 'No', 'driver'),
27 (12, 'No', 'driver'),
28 (13, 'No', 'driver');
29
30 INSERT INTO Trips VALUES
31 (1, 1, 10, 1, 'completed', '2013-10-01'),
32 (2, 2, 11, 1, 'cancelled_by_driver', '2013-10-01'),
33 (3, 3, 12, 6, 'completed', '2013-10-01'),
34 (4, 4, 13, 6, 'cancelled_by_client', '2013-10-01'),
35 (5, 1, 10, 1, 'completed', '2013-10-02'),
36 (6, 2, 11, 6, 'completed', '2013-10-02'),
37 (7, 3, 12, 6, 'completed', '2013-10-02'),
38 (8, 2, 12, 12, 'completed', '2013-10-03'),
```



```

39 (9,3,10,12,'completed','2013-10-03'),
40 (10,4,13,12,'cancelled_by_driver','2013-10-03');

```

Answer

```

1  SELECT
2  t.request_at AS day,
3
4  ROUND(
5      SUM(CASE WHEN t.status <> 'completed' THEN 1 ELSE 0 END)::NUMERIC
6      / COUNT(*)
7  , 2) AS cancellation_rate
8
9  FROM Trips t
10
11 JOIN Users c
12 ON t.client_id = c.users_id AND c.role = 'client'
13
14 JOIN Users d
15 ON t.driver_id = d.users_id AND d.role = 'driver'
16
17 WHERE c.banned = 'No'
18 AND d.banned = 'No'
19 AND t.request_at BETWEEN '2013-10-01' AND '2013-10-03'
20
21 GROUP BY t.request_at
22
23 ORDER BY t.request_at;

```

Output

day	cancellation_rate
2013-10-01	0.33
2013-10-02	0.00
2013-10-03	0.50

Learning Outcome

After completing this experiment, the following learning outcomes were achieved.

- Understood how to compute rate/ratio metrics using conditional aggregation with **CASE** inside **SUM()**.
- Applied **LEFT JOIN** to include entities with zero related events, and used **COALESCE** with **NULLIF** to safely handle division by zero.
- Learned to enforce multi-condition validity checks (banned status of two related roles) using multiple joins with role-specific conditions.
- Applied date-range filtering with **BETWEEN** to restrict analysis to a specific reporting window.
- Used **GROUP BY** with **ROUND()** to produce day-wise aggregated business metrics.



- Gained practical exposure to solving real-world platform analytics problems (confirmation rates and cancellation rates) using PostgreSQL.